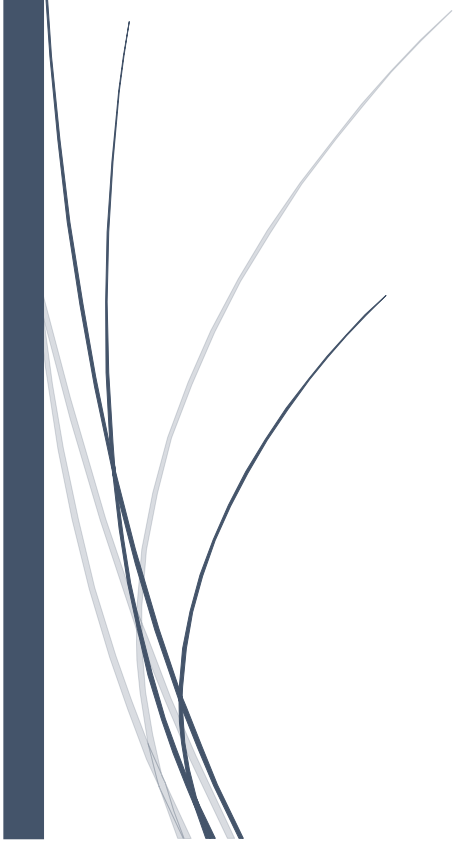
CLASES

PROGRAMACIÓN
TEMA 6



ÍNDICE

INTRODUCCIÓN.....3
OBJETOS.....3
CLASES.....4
MENSAJES.....5
DEFINICIÓN DE UNA CLASE.....5
ATRIBUTOS.....6
INICIALIZACIÓN DE LOS ATRIBUTOS.....6
OBJETOS.....6
A. REFERENCIAS.....7
B. VARIABLES REFERENCIA.....7
C. OPERADOR NEW.....7
D. REFERENCIA null.....8
E. RECOLECTOR DE BASURA.....8
MÉTODOS.....9
A. ÁMBITO DE LAS VARIABLES Y ATRIBUTOS.....9
B. OCULTACIÓN DE ATRIBUTOS.....10
C. OBJETO this.....10
ATRIBUTOS Y MÉTODOS ESTÁTICOS.....10
CONSTRUCTORES.....11
A. CONSTRUCTOR this().....11
PAQUETES.....12
MODIFICADORES DE ACCESO.....12
A. MODIFICADORES DE ACCESO PARA CLASES.....12
B. MODIFICADORES DE ACCESO PARA MIEMBROS.....13
C. MÉTODOS GET/SET.....14
ENUMERADOS.....14
WRAPPERS.....14

INTRODUCCIÓN

Hasta aquí hemos utilizado un paradigma de programación llamado programación estructurada, que emplea las estructuras de control (condicionales y bucles), junto a datos y funciones para la resolución de los problemas planteados. Una de sus principales desventajas es que no existe un vínculo fuerte entre funcionalidad y datos. Esto dificulta el tratamiento de problemas complejos.

Por eso, llegados a este punto, vamos a saltar a un nuevo paradigma, la programación orientada a objetos (en adelante POO), que amplía los horizontes de un programador, dotándolo de nuevas herramientas que facilitan la resolución de problemas complejos.

OBJETOS

Objeto es el concepto clave de la Programación Orientada a Objetos: la idea de objeto es similar a la del mundo real, un objeto puede ser una silla, una mesa. Un perro puede modelarse como un objeto.

Los objetos son definidos en POO por dos de sus características: el estado (almacenado en los atributos) y el comportamiento (descrito mediante los métodos/funciones de un objeto). Por ejemplo, un objeto perro tiene un estado: su nombre concreto, su color, su raza, su altura, etc. y un comportamiento: sabe ladrar, puede cavar un pozo, sabe gemir, dormir, comer, etc. Un coche es otro ejemplo de objeto. También tiene un estado: Cantidad de puertas, color, tamaño, tipo de motor, etc. y un comportamiento: acelerar, frenar, subir cambio, bajar cambio, girar izq., girar der., etc.

Podemos definir a un objeto en POO, como un conjunto de datos y funciones relacionadas que modelan una entidad del mundo determinada. Una entidad puede ser algo que tenga existencia física real o imaginada, pues podemos imaginar un triángulo y también sería un objeto aunque físicamente no lo tengamos con nosotros. A las funciones de los objetos, tales como acelerar en el caso del coche, de aquí en adelante las llamaremos "métodos", mientras que a los datos del estado los llamaremos "atributos". Es decir, los estados son los valores de los distintos atributos de un objeto, y los métodos son los que definen que “sabe” hacer ese objeto, es decir, su comportamiento.

Los objetos en programación son modelados observando objetos del mundo real, por ejemplo, implementamos el objeto "perro" dentro de nuestro programa definiendo los atributos y métodos del objeto perro real que nos interese representar. Nuestro objeto modelado tendrá aquellos atributos y métodos que nos interesen para nuestro proposito concreto, pues en el fondo, todo objeto tiene infinitos posibles atributos.

El objeto "Coche" y el objeto "Cuenta bancaria" se pueden representar como sigue:

Objeto: Coche de mi padre

Atributos: matrícula, tamaño, color, puertas, tipo de motor, marca, modelo

Métodos: Arrancar, Apagar, Acelerar, Frenar, Girar, CambiarMarcha

Objeto: Cuenta bancaria de mi hermano

Atributos: tipo, titular, saldo.

Métodos: Depositar, Extraer, Cerrar.

Cómo se puede intuir, un objeto "almacena" información en los atributos y muestra información, cambia su estado y realiza acciones mediante los métodos. Tanto los atributos como los métodos de un determinado objeto pueden ser "públicos" o "privados", que significa que están con acceso libre o limitado, por razones obvias, como se puede observar en el objeto "Cuenta bancaria", en el que interesa que el atributo saldo, por ejemplo, sea privado.

CLASES

¿Qué es una clase? Una clase es la abstracción que define la "forma" de unos determinados objetos que comparten los tipos de atributos que poseen y sus comportamientos, se podría hablar de la clase como el molde de los objetos. Por ejemplo, la clase "Mesa" es la abstracción de muchas mesas, obteniendo las características comunes de todas ellas (tanto en sus atributos como en su comportamiento).

Otro ejemplo podría ser tu bicicleta: es solo una más de todas las bicicletas del mundo. Entonces diríamos que tu bicicleta es una instancia de la clase "Bicicleta", que es lo mismo que decir que tu bicicleta es un objeto "Bicicleta". Todas las bicicletas tienen los atributos: color, cantidad de cambios, dueño; y los métodos: acelerar, frenar, pasar cambio, volver cambio, eso sí, tu bicicleta es la que tiene el color azul, 18 cambios, tu eres el dueño, frena pedaleando hacia atrás pues tiene el sistema de frenado en pedales, etc. Eso sí, todas las bicicletas, parecidas o no a la tuya, tendrán un color, un dueño y un número de cambios. Esta abstracción es la que representa la clase Bicicleta.

Las fabricas de bicicletas utilizan moldes para producir sus productos en serie, de la misma forma en POO utilizaremos la clase Bicicleta (molde) para producir sus instancias bicicletas (objetos). Por lo tanto, los objetos son instancias de clases previamente definidas.

Ejemplo: Clase Coche y objeto coche.

La clase coche podría tener:

Atributos: matrícula, tamaño, color, puertas, tipo de motor, marca, modelo

Métodos: Arrancar, Apagar, Acelerar, Frenar, Girar, CambiarMarcha

El objeto coche "1489-CFG" sería:

Matrícula: 1489-CFG

Tamaño: 4,50m

Color: Blanco

Puertas: 5

Tipo de motor: Gasolina 1500cc

Marca: Renault

Modelo: Megane

Podemos definir a una clase como una plantilla que define atributos y métodos comunes para todos los objetos de cierto tipo.

MENSAJES

Un objeto por sí solo no tiene mucho sentido. Ejemplo: el objeto "bicicleta" no tiene mucho sentido si no interactúa con un objeto "persona" que pedalee. La interacción entre objetos se produce mediante mensajes, y esta es la clave del dinamismo de nuestros programas cuando usamos los objetos. Los mensajes se producen cuando desde un método de un objeto A se utiliza un método de un objeto B. Por ejemplo, si dentro del método "ir_en_bici" de la clase Persona utilizamos el método "pedalear" de la clase Bicicleta, se dice que Persona manda un mensaje a Bicicleta. Otro ejemplo sería: el objeto persona envía el mensaje "girar a la izquierda" al objeto bicicleta.

Los mensajes (métodos) pueden contener parámetros. Por ejemplo: un método en la clase bicicleta llamado "Girar" que recibe como parámetro la dirección (derecha o izquierda). En esto son exactamente lo mismo que las funciones estudiadas en temas previos, unicamente con la diferencia que las funciones son “estancas” e independientes teniendo en cuenta sólo la entrada y la salida de la propia función, y sin embargo un método de una clase es una función aplicada a su entrada y a los estados contenidos en sus atributos.

Un mensaje esta compuesto por los siguientes tres elementos:

- El objeto destino, hacia el cual el mensaje es enviado
- El nombre del método a llamar
- Los parámetros solicitados por el método

Ejemplo: Objeto impresoralaser, de la clase Impresora, con un método ImprimirDocumento que necesita dos parámetros, el texto a imprimir y si es a color o en ByN. Otro objeto Editor, podría mandar el siguiente mensaje:

impresoralaser.ImprimirDocumento(”Esto saldrá impreso”,color)

DEFINICIÓN DE UNA CLASE

La POO se inspira en una abstracción del mundo real, en la que los objetos se clasifican en grupos, que denominamos clases, tal y cómo hemos visto en los párrafos previos.

Podemos definir cada grupo o clase mediante las propiedades y comportamientos que presentan todos sus miembros. Una propiedad es un

dato que conocemos de cada miembro del grupo, mientras que un comportamiento es algo que puede hacer.

Vamos a definir la clase Persona mediante dos elementos:

- Propiedades, también llamadas atributos. Un nombre, una edad y una estatura. Tanto Pepa como Paco como Miguel tienen un nombre, una edad y una estatura; son datos que en cada uno tendrán un valor distinto. A estos valores, los denominamos "estado" del atributo. El nombre de los atributos no cambiará, pero su estado, probablemente sí.
- Comportamientos, también denominados métodos o funciones. Pepa, Paco y Miguel, en realidad todas las personas, pueden saludar, crecer o cumplir años, comunicarse, por ejemplo.

Como vemos es posible definir una clase mediante un conjunto de propiedades y comportamientos. La sintaxis para definir una clase usa la palabra reservada class.

ATRIBUTOS

Las propiedades que definen una clase se denominan, como hemos visto, atributos. Por ejemplo, la clase Vehículo puede definirse mediante los atributos matrícula, color, marca y modelo. Como hemos visto antes, nuestra clase Persona, dispone de los atributos nombre, edad y estatura.

Los atributos de una clase serían las "variables" que posee un objeto de esta clase, las que puede asignar para definir su estado. Por lo que en su declaración tenemos que escoger un tipo de dato para cada atributo (int, String, float, double, boolean, etc.), y su publicidad que será "por defecto" cuando no ponemos nada, public, private o protected (veremos más adelante que significan estas palabras reservadas y que consecuencias tienen cuando se aplican a un atributo).

INICIALIZACIÓN DE LOS ATRIBUTOS

Como cualquier otra variable, se puede asignar un valor por defecto a los atributos de una clase. Esto se realiza en la propia declaración del atributo.

En general, los atributos pueden cambiar durante la ejecución de un programa, como variables que son, proporcionando dinamismo al objeto, salvo que sean declarados como constantes (modificador *final*). Por ejemplo, el atributo edad de una clase Persona cambiará año tras año. Sin embargo, la fecha de nacimiento no debe de cambiar, tendría que ser constante.

La inicialización de un atributo constante puede hacerse en su declaración o por medio de un constructor.

OBJETOS

Los elementos que pertenecen a una clase se denominan instancias u objetos. Cada uno tiene sus propios valores para los atributos definidos en la clase. Es lo que llamamos "estado" del objeto. Estos objetos son declarados como variables, atributos, o cualquier otra referencia en nuestros programas. Posteriormente

son inicializados (creados) y cuando un objeto deja de estar referenciado (no hay ninguna referencia que lo apunte en RAM) es reciclado por el colector de basura de la máquina virtual Java. Veamos en detalle cada aspecto de lo que acabamos de describir.

A. REFERENCIAS

El comportamiento de los objetos en la memoria del ordenador y sus operaciones elementales (creación, asignación y destrucción) son idénticos al de las tablas. Esto es debido a que ambos, objetos y tablas, utilizan las referencias. De hecho, las propias tablas son en java un tipo más de objetos.

Cualquier dato almacenado en la memoria ocupará, dependiendo de su tamaño, una serie de bloques consecutivos, y puede ser identificado mediante la dirección del primero de ellos. A esta primera dirección de memoria que identifica un objeto se le denomina en Java referencia.

Por lo tanto, si un objeto declara en sus atributos dos int y un double, por ejemplo, una instancia o referencia de nuestro objeto apuntará al lugar de la RAM donde esté el primer int, y el otro int más el double estáran justo a continuación. El tamaño total será la suma de bytes de los dos int más los bytes del double, como cada int ocupa 4 bytes y un double son 8 bytes, nuestro objeto ocupará 16 bytes en total.

B. VARIABLES REFERENCIA

Antes de construir un objeto necesitamos declarar una variable cuyo tipo sea su clase. La declaración sigue las mismas reglas que las variables de tipo primitivo, pero en vez de poner el tipo primitivo, ponemos el nombre de la clase. Es importante que la clase a utilizar sea del mismo paquete de la clase que "declara" la variable referencia (nuestro objeto), o en otro caso, que sea importada mediante la clase *import* java.

La diferencia entre una variable de tipo primitivo y una variable de tipo referencia es que mientras una variable de tipo primitivo almacena directamente un valor, una variable del tipo clase almacena la referencia de un objeto.

C. OPERADOR NEW

La forma de crear objetos, como en las tablas, es mediante el operador *new* java.

El operador *new* primero busca en memoria un hueco disponible donde construir el objeto. Este, dependiendo de su tamaño, ocupará cierto número consecutivo de bloques de memoria. Por último, devuelve la referencia del objeto recién creado, que se asigna a la variable creada.

El tamaño de los objetos de una clase depende del tipo y la cantidad de atributos que esta contenga, como hemos indicado a la hora de definir lo

que es un objeto. Por eso decimos que una clase es el "molde" para fabricar objetos de esa clase.

A la hora de trabajar con referencias, es bastante más sencillo pensar en ellas como flechas que se dirigen desde la variable hacia el objeto en la RAM.

En el momento en el que disponemos de un objeto, podemos acceder a sus atributos mediante el nombre de la variable seguido de un punto aunque sólo funcionará para atributos con la clausula *public* o *protected*. De todas formas, usar la clausula *public* para atributos veremos cuando diseñemos clases que es un error de diseño, por lo que declararemos prioritariamente los atributos como *private* (o *protected* en algunos casos particulares). Nosotros en nuestras NUNCA declararemos atributos como *public*.

Es importante comprender que podemos acceder al mismo objeto mediante distintas variables que almacenan la misma referencia. El mecanismo en el que varias variables comparten la misma referencia es aprovechado por la clase String para ahorrar espacio en textos usados frecuentemente, ya que Java se encarga por su cuenta de que todas las variables a las que se les han asignado idéntico literal cadena compartan su referencia.

El hecho de que se compartan las referencias de un mismo literal cadena es la causa de que la clase sea inmutable (un objeto de la clase String nunca cambia su contenido realmente).

D. REFERENCIA null

El valor literal "*null*" es una referencia nula. Es decir, es una referencia a ningún bloque de memoria. Cuando declaramos una variable referencia se inicializa por defecto a "*null*". Es la forma que tiene Java de inicializar las variables o atributos no inicializados distintos a los tipos primitivos. Igual que un int se inicializará por defecto al valor 0, una variable o atributo de la clase Persona por defecto estará inicializado a null.

Hay que tener mucho cuidado de no intentar acceder a los miembros de una referencia nula, ya que se produce un error que termina la ejecución de forma inesperada (Se trata de una Runtime exception, que las veremos con detalle más adelante)

El valor "null" se puede asignar a cualquier variable referencia, aunque sean de clases diferentes.

E. RECOLECTOR DE BASURA

Existen tres formas de conseguir que un objeto no esté referenciado:

1. Creándolo y no asignándolo a ninguna variable. Es una situación atípica pero a veces quizás necesitemos crear un objeto y llamar justo

de manera instantanea a alguno de sus métodos, tras lo cual no se necesite más.
2. Asignándoles el valor "null" a todas las variables que contenían una referencia a ese objeto.
3. Asignando todas las variables que referenciaban a un objeto a objetos diferentes del que dejamos "huerfano".

En todos los casos, el objeto se queda perdido en memoria, es decir, no existe forma de acceder a él. Sin embargo, está ocupando memoria. Si este comportamiento se repite demasiado, fortuita o malintencionadamente es posible que se agote toda la memoria libre disponible, lo que impediría el normal funcionamiento del ordenador.

Para evitar este problema, Java dispone de un mecanismo llamado, recolector de basura, "garbage collector", que se ejecuta de vez en cuando de forma transparente al usuario, y se encarga de comprobar uno a uno, todos los objetos de la memoria. Si alguno de ellos no estuviera referenciado por ninguna variable, se destruye, liberando la memoria que ocupa.

MÉTODOS

Hemos declarado clases con atributos, pero también disponen de comportamientos. En el argot de la POO, a los comportamientos u operaciones que pueden realizar los objetos de una clase se les denomina **métodos**. Por ejemplo, las personas son capaces de realizar operaciones como saludar, cumplir años, crecer, etc.

Los métodos no son más que funciones que se implementan dentro de una clase. Estos métodos/funciones se encapsulan dentro de la clase para que usen exclusivamente como entorno sus parámetros de entrada y los valores de los atributos del objeto que recibe el encargo de ejecutar el método en cuesttión.

Tanto a los atributos como a los métodos de una clase se les llama de forma genérica miembros. De esta forma, al hablar de miembros de una clase, hacemos referencia a los atributos y métodos declarados en su definición.

Los métodos de una clase tienen acceso a las siguientes variables: variables locales declaradas dentro del método, parámetros de entrada y atributos de la clase. Así mismo, tiene acceso a los demás métodos de la clase.

A. ÁMBITO DE LAS VARIABLES Y ATRIBUTOS

El ámbito de una variable define en qué lugar puede usarse y coincide con el bloque en el que se declara la variable que puede ser:

- El bloque de una estructura de control que dará lugar a variables de bloque.
- Una función o método que dará lugar a variables locales.

Con la POO, aparece un nuevo ámbito: la clase que dará lugar a variables de la clase.

Un ámbito puede contener a otros ámbitos, formando una estructura jerárquica. Por ejemplo, una clase puede contener dos métodos, y estos, distintos bloques de, por ejemplo, una estructura while o if.

Una variable puede utilizarse en el ámbito o bloque en el que se declara, que incluye sus bloques internos. Sin embargo, no ocurre lo contrario: una variable no podrá utilizarse en el ámbito padre del bloque en el que se declara.

Por ejemplo, un atributo puede emplearse dentro de un método, y una variable local dentro del bloque de una estructura de control de un método. Pero, en cambio, no podemos usar una variable local fuera de su método, ni una variable de bloque fuera de él.

B. OCULTACIÓN DE ATRIBUTOS

Dos variables declaradas en ámbitos anidados no pueden tener el mismo identificador, ya que esto genera un error. Sin embargo, existe una excepción cuando una variable local en un método tiene el mismo identificador que un atributo de la clase o variable que esté declarada en un ámbito jerárquicamente superior. En este caso, dentro del método, la variable local tiene prioridad sobre el atributo, es decir, que al utilizar el identificador se accede a la variable local y no al atributo. En la jerga de la POO se dice que la variable local oculta al atributo. Esto es algo que tenemos que evitar, pues complica innecesariamente el código y su comprensión dando lugar tarde o temprano a errores muy difíciles de localizar y corregir.

C. OBJETO this

La palabra reservada *this* permite utilizar un atributo incluso cuando ha sido ocultado por la variable local o parámetro de un método. De igual manera que cada uno se refiere a sí mismo como yo, aunque tengamos un nombre que los demás utilizan para identificarnos, las clases se refieren a sí mismas como *this*, que es una referencia al objeto actual y funciona como una especie de "yo" para las clases.

Al escribir *this* en el ámbito de una clase se interpreta como la propia clase, y permite acceder a los atributos aunque se encuentren ocultos.

ATRIBUTOS Y MÉTODOS ESTÁTICOS

Un atributo estático, también llamado atributo de la clase, es aquel del que no existe una copia en cada objeto. Todos los objetos de una misma clase comparten su valor. Por eso también se denomina atributo de "clase".

Un atributo estático se inicializará en el momento de cargar la clase en memoria; esto ocurre cuando se declara alguna variable del tipo de la clase o cuando se crea un primer objeto de dicha clase.

También podemos declarar métodos estáticos. Son aquellos que no requieren de ningún objeto para ejecutarse y, por tanto, no pueden utilizar ningún atributo

que no sea estático. En el caso de que se intente, se producirá un error. La forma de invocar un método estático es mediante el nombre de la clase, justo delante del punto y del nombre del propio método.

Por otra parte, desde un método estático sólo se pude invocar directamente métodos y atributos estáticos (pues al ser de la clase puede que incluso no existan en memoria RAM, en el momento de la ejecución, ningún objeto de la clase)

No obstante, dentro de un método estático se pueden crear objetos de cualquier clase, incluida la suya propia, y desde él invocar miembros no estáticos definidos en esta clase usando estas referencias.

CONSTRUCTORES

Es un método especial que debe tener el mismo nombre que la clase, se define sin tipo devuelto (ni siquiera void), y se ejecuta inmediatamente después de crear el objeto. El principal cometido de un constructor es asignar valores a los atributos, aunque también se puede utilizar para otros fines como crear tablas, crear otros objetos que necesitemos, etc.

Al constructor como cualquier otro método, se le pueden pasar parámetros y se puede sobrecargar. La llamada al constructor con los valores de los parámetros de entrada se hace por medio del operador *new* en el proceso que necesitemos usar este nuevo objeto.

Los atributos declarados como final también se pueden inicializar pasando sus valores como parámetros al constructor; no es necesario hacerlo en el sitio donde se declaran.

A la hora de sobrecargar un método tenemos que asegurarnos de que se puede distinguir entre las distintas versiones mediante el número o el tipo de parámetros de entrada. La sobrecarga de constructores es útil cuando necesitamos inicializar objetos de varias formas.

Cuando en una clase no se implementa ningún constructor, Java se encarga de crear uno que se denomina constructor por defecto. Este no usa parámetros de entrada e inicializa los atributos a cero, false o null según el tipo si no están ya inicializados en su declaración.

No obstante, es conveniente implementar los constructores y no dejarlo en manos de Java. En cuanto se implementa un constructor en una clase, el constructor, por defecto, deja de estar disponible.

A. CONSTRUCTOR this()

Cuando una clase dispone de un conjunto de constructores sobrecargados, es posible que un constructor invoque a otro y así reutilice su funcionalidad. Para eso se usa el constructor genérico this(), en lugar del constructor por su nombre.

La forma de distinguir los distintos constructores, igual que en cualquier método sobrecargado, es mediante el número y el tipo de los parámetros de entrada.

Tenemos que tener presente que, en caso de utilizar this(), tiene que ser siempre la primera instrucción de un constructor; en otro caso se producirá un error.

PAQUETES

En Java es importante controlar la accesibilidad de unas clases desde otras por razones de seguridad y eficiencia. Esto se consigue mediante paquetes, que son contenedores que permiten guardar clases en compartimentos separados, de modo que podamos decidir, por medio de la importación, qué clases son accesibles y cómo se accede a ellas desde una clase que estemos implementando.

Todas las clases están dentro de algún paquete que, a su vez, pueden estar anidados unos dentro de otros. Se considera que una clase que pertenece a un paquete, que a su vez está dentro de otro, sólo pertenece al primero, pero no al segundo.

Sabemos que un archivo de código fuente Java es un archivo de texto con extensión ".java", que se guarda en un paquete (al final nos daremos cuenta que los IDE's crean un subdirectorio por cada "subpaquete") y que contiene los siguientes elementos:

- Una sentencia donde se especifica el paquete al que pertenece, que empieza con la palabra clave ***package*** seguida del nombre del paquete.
- Una serie opcional de sentencias de importación, con la palabra reservada **import**, que permite importar clases definidas en otros paquetes.
- La definición de una o más clases, de las cuales sólo una puede ser declarada pública, por medio del modificador de acceso **public**. De todas formas, es recomendable que en cada archivo fuente se defina una sola clase, que debe tener el nombre del archivo.

MODIFICADORES DE ACCESO

Una clase será visible por otra, o no, dependiendo de si se ubican en el mismo paquete y de los modificadores de acceso que utilice. Estos modifican su visibilidad, permitiendo que se muestre u oculte.

De igual manera que podemos modificar la visibilidad entre las clases, es posible modificar la visibilidad entre los miembros de distintas clases, es decir, qué atributos y métodos son visibles para otras clases.

A. MODIFICADORES DE ACCESO PARA CLASES

Debido a la estructura de clases, organizadas en paquetes, que utiliza Java, dos clases cualesquiera pueden definirse de las siguientes formas:

- Clases vecinas: pertenecen al mismo paquete.

- Clases externas: pertenecen a paquetes distintos.

Existen varios grados de visibilidad:

- Visibilidad por defecto: Cuando se define una clase sin ningún modificador de acceso hace que sólo sea visible desde clases vecinas.
- Visibilidad total: Cuando se utiliza el modificador public la clase además de ser visible por sus clases vecinas, lo será por las clases externas previa importación.

B. MODIFICADORES DE ACCESO PARA MIEMBROS

De igual manera que es posible modificar la visibilidad de una clase, podemos regular la visibilidad de sus miembros. Que un atributo sea visible significa que podemos acceder a él, tanto para leer como para modificarlo. Que un método sea visible significa que puede ser invocado.

Para que un miembro sea visible, es indispensable que su clase también lo sea. Es evidente que, si no podemos acceder a una clase, no existe forma de acceder a sus miembros.

Debemos destacar que cualquier miembro es siempre visible dentro de su propia clase, indistintamente del modificador de acceso que utilicemos. Es decir, desde dentro de la definición de una clase siempre tendremos acceso a todos sus atributos y podremos invocar cualquiera de sus métodos.

- Visibilidad por defecto. Cuando queramos acceder a miembros de otra clase hay diversos grados de visibilidad. La visibilidad por defecto es aquella que se aplica a miembros declarados sin ningún modificador de acceso.

  La visibilidad por defecto hace que un miembro sea visible desde las clases vecinas, pero invisible desde clases externas (es como un "public" pero sólo para clases del mismo paquete.

- Modificador de acceso private y public. Con el modificador de acceso private obtenemos una visibilidad más restrictiva que por defecto, ya que impide el acceso para las clases vecinas. Un miembro, ya sea un atributo o un método, declarado privado, es invisible desde fuera de la clase.

  En cambio, public hace que un miembro sea visible incluso desde clases externas previa importación. Otorga visibilidad total.

  Usaremos private cuando queramos controlar los cambios de un atributo o cuando deseamos que no se conozca directamente su valor, o bien cuando queremos que un método sea invocado desde otros métodos de la clase, pero no fuera de ella.

El acceso a los miembros privados deberá hacerse a través de algún método public de la misma clase (en los casos de los atributos, mediante los métodos getters y setters).

Es importante aprender que por norma general, siempre pondremos los atributos como private, y los métodos como public. Debemos de evitar a toda costa poner los atributos de una clase con visibilidad public pues esto puede hacer que no se cumplan algunos de los principios básicos de un buen diseño software (por ejemplo los principios SOLID, que veremos en el curso más adelante).

C. MÉTODOS GET/SET

Un atributo público puede ser modificado desde cualquier clase, lo que tiene sus inconvenientes, ya que es imposible controlar los valores asignados, que pueden no tener sentido.

Por este motivo, existe una convención en la comunidad de programadores que consiste en ocultar los atributos de las clases y, en su lugar, crear dos métodos públicos: el primero, habitualmente llamado set, permite asignar un valor al atributo, controlando el rango válido de valores. El segundo, habitualmente llamado get, devuelve el atributo, lo que posibilita conocer su valor. Los métodos set/get hacen que un atributo no visible se comporte como si lo fuera pero con un control adecuado de la lógica de uso por parte del programador.

Las ventajas de utilizar métodos set/get son que la implementación de la clase se encapsula, ocultando los detalles y, por otro lado, permite controlar qué atributos son accesibles para lectura y cuáles para escritura, así como los valores asignados.

ENUMERADOS

Los tipos enumerados sirven para definir grupos de constantes como posibles valores de una variable.

Se definen de forma parecida a una clase pero utilizando la palabra clave enum y no class. En un programa se accede a sus valores de la forma nombreEnumerado.valorEnumerado (por ejemplo, DiaDeLaSemana.LUNES).

Un tipo enumerado se puede implementar en un archivo aparte, normalmente dentro del mismo paquete, aunque no es obligatorio, como si fuera una clase o bien dentro de la definición donde se va a usar.

Los tipos enumerados se pueden definir en paquetes distintos a donde se vayan a usar. En este caso, habrá que definirlos public e importarlos igual que si fueran clases.

WRAPPERS

También conocidos como envoltorios son clases que internamente contienen un dato de tipo primitivo, lo que proporciona una forma de trabajar con estos tipos

primitivos como objetos. Cada tipo primitivo tiene su correspondiente clase envoltorio: Integer para los int, Double para los double, Character para los char, etc.

Serán de mucha utilidad cuando estudiemos los tipos avanzados de datos, como las colecciones.